Системы компьютерной обработки изображений

Лабораторная работа № 4

«Формирование распределения случайных величин в заданной области определения»

Цель

Изучить методы формирования заданных распределений случайных координат и направлений лучей в пространстве. Доказать корректность полученных распределений случайных величин.

Задание

1. Сформировать равномерное распределение случайных точек (P_1, P_2, …) внутри треугольника. Треугольник задан тремя координатами в пространстве (V_1, V_2, V_3). Число точек используемых в выборке 100000, число точек, сформированных внутри треугольника также 100000.

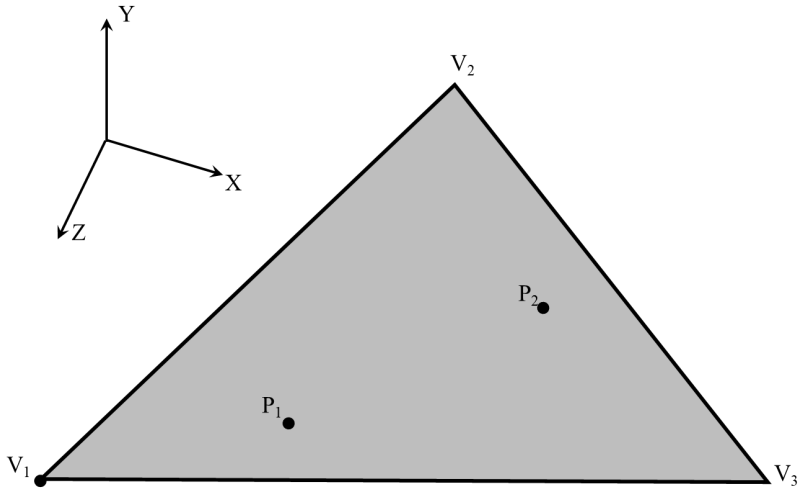


Рис. 1.1 Задание 1

Доказать, что полученное распределение равномерное и все точки лежат внутри треугольника.

Решение:

Сгенерируем точки, используя следующий алгоритм (рис. 1.2)

- С помощью функции rand сформируем два набора координат u и v в диапазоне [0, 1]. Получаем квадрат из точек (голубой на рисунке).

- Точки, выходящие за диагональ 1-1 «переотражаем» вниз (u=1-u, v=1-v). Получаем треугольник из точек (зеленый).
- Деформируем треугольник в координаты P0, P1, P2

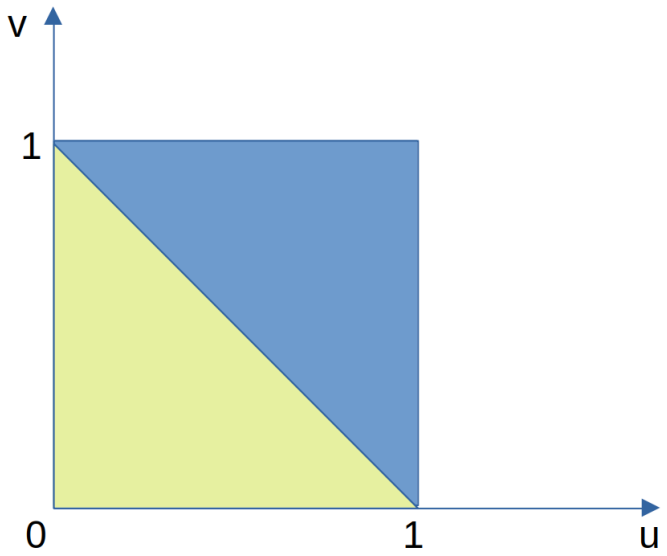


Рис. 1.2 Алгоритм генерации точек

Листинг 1.1Код генерации точек

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------
# Треугольник в пространстве (любые координаты)
# ------------------------------------
V1 = np.array([0, 0, 0])
V2 = np.array([2, 1, 0.5])
V3 = np.array([-1, 2, 1])

N = 100000   # число точек

# ------------------------------------
# Функция генерации равномерных точек внутри треугольника
# Метод: u,v — независимые равномерные, но если u+v>1 – отражаем
# ------------------------------------
def sample_triangle(V1, V2, V3, n):
    u = np.random.rand(n)
```

```
    v = np.random.rand(n)

    # Отражение для равномерности
    mask = u + v > 1
    u[mask] = 1 - u[mask]
    v[mask] = 1 - v[mask]

    # Генерация точек
    return V1 + u[:,None]*(V2 - V1) + v[:,None]*(V3 - V1)

# Сгенерируем точки
points = sample_triangle(V1, V2, V3, N)

# ---------------------------------------
# Чтобы визуализировать распределение, надо перевести точки в 2D
# (система координат треугольника)
# ---------------------------------------
e1 = V2 - V1
e2 = V3 - V1

# базис (ортонормируем)
e1n = e1 / np.linalg.norm(e1)
e2p = e2 - np.dot(e2, e1n) * e1n
e2n = e2p / np.linalg.norm(e2p)

# Проекция на 2D
proj_x = np.dot(points - V1, e1n)
proj_y = np.dot(points - V1, e2n)

# ---------------------------------------
# Визуализация тепловой карты распределения
# ---------------------------------------
plt.figure(figsize=(6, 6))
plt.hist2d(proj_x, proj_y, bins=100, density=True)
plt.colorbar(label="Плотность")
plt.title("Тепловая карта плотности точек")
plt.xlabel("X (local)")
plt.ylabel("Y (local)")
plt.show()

# ---------------------------------------
```

```
# Покажем треугольник и первые 500 точек
# ------------------------------------
tri_proj = np.array([
    [np.dot(V1 - V1, e1n), np.dot(V1 - V1, e2n)],
    [np.dot(V2 - V1, e1n), np.dot(V2 - V1, e2n)],
    [np.dot(V3 - V1, e1n), np.dot(V3 - V1, e2n)],
])

plt.figure(figsize=(6, 6))
plt.fill(tri_proj[:,0], tri_proj[:,1], edgecolor='black', fill=False, linewidth=2)

plt.scatter(proj_x[:500], proj_y[:500], s=10, color='red')
plt.title("Треугольник и первые 50 точек")
plt.xlabel("X (local)")
plt.ylabel("Y (local)")
plt.gca().set_aspect('equal')
plt.show()
```

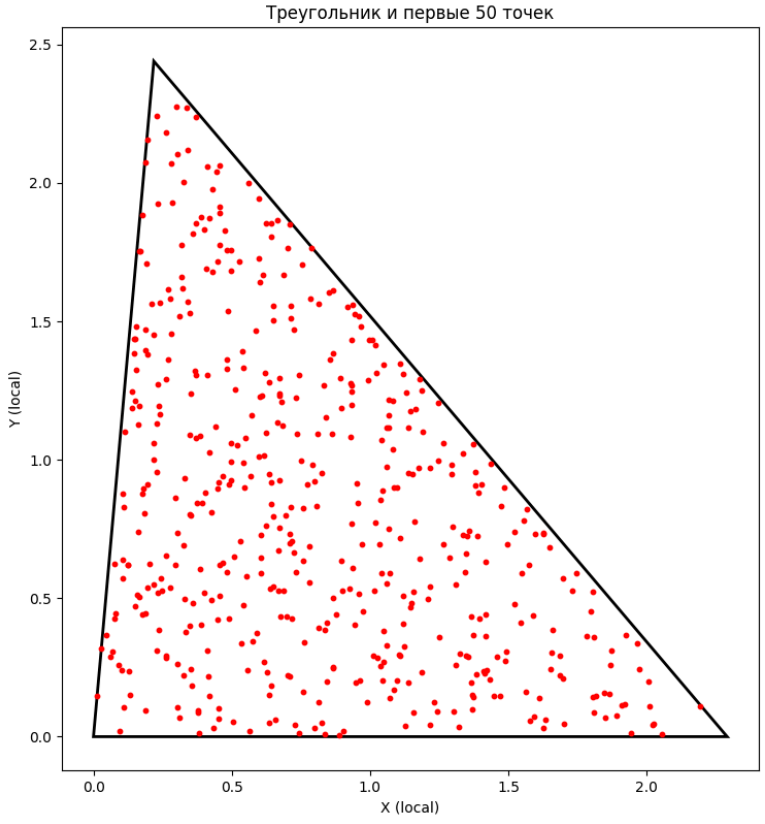


Рис. 1.3 Визуализация треугольника (2D – проекция)

Для проверки равномерности построим тепловую карту распределения. Для этого:

🎬 Разобьём треугольник на регулярную сетку, например 100×100 ячеек, и посчитаем, сколько точек попало в каждую ячейку.

🎬 Построим тепловую карту плотности точек (в 2D-проекции) (рис. 1.4)

По гистограмме видно, что зон с аномальным количеством точек нет – распределение равномерное.

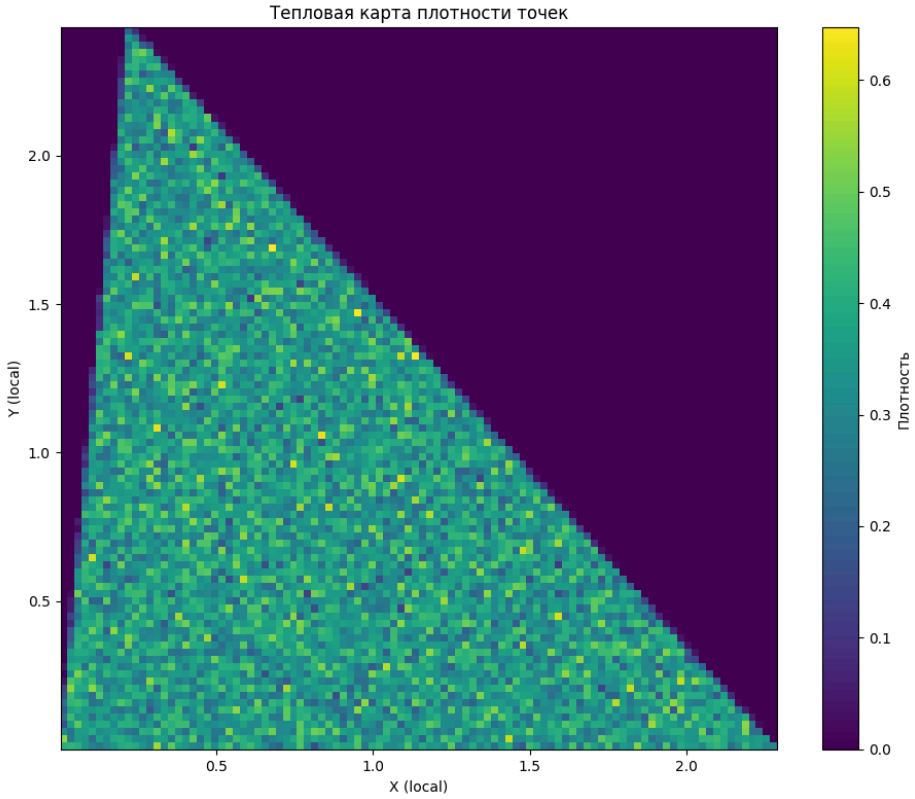


Рис. 1.4 Карта распределения плотности точек

2. Сформировать равномерное распределение случайных точек (P_1, P_2, …) внутри круга. Круг задан ориентацией нормали N, радиусом круга R_c, и его центром C. Число точек используемых в выборке 100000, число точек, сформированных внутри круга также 100000.

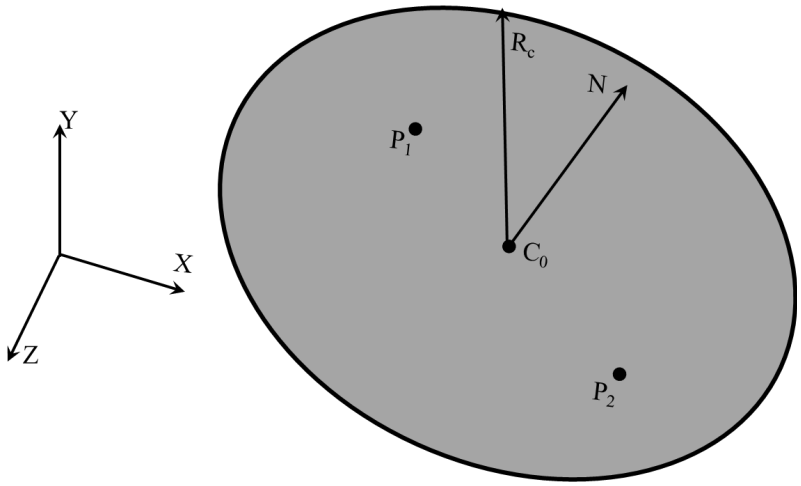


Рис. 2.1 Задание 2

Доказать, что полученное распределение равномерное и все точки лежат внутри круга.

Решение:

Генерируем равномерно распределенные точки внутри диска, используя формулы:

$$r = R * \sqrt{u} \text{ и } \theta = 2 * \pi * u$$

Где

R – максимальный радиус;

u – случайное число [0, 1];

Затем переводим их в локальные координаты через sin и cos угла и гипотенузу.

Листинг 2.1 Код генерации точек

```
import numpy as np
```

```
import matplotlib.pyplot as plt

# Исходные параметры
C = np.array([1.0, 2.0, 0.5])   # центр окружности в 3D
N_vec = np.array([0.3, -0.6, 0.74])  # вектор нормали (ориентация)
Rc = 1.25                       # радиус
N_points = 100000   # количество точек

def normalize(v, eps=1e-12):
    n = np.linalg.norm(v)
    if n < eps:
        raise ValueError("Вектор нулевой длины")
    return v / n

# Рассчитываем ортогональный базис (u_axis, v_axis) для
плоскости круга
Nn = normalize(N_vec)
if abs(Nn[0]) < 0.9:
    tmp = np.array([1.0, 0.0, 0.0])
else:
    tmp = np.array([0.0, 1.0, 0.0])
u_axis = normalize(np.cross(Nn, tmp))
v_axis = normalize(np.cross(Nn, u_axis))

# Генерируем равномерно распределенные точки внутри диска
u = np.random.rand(N_points)
theta = 2 * np.pi * np.random.rand(N_points)
r = Rc * np.sqrt(u)
x_local = r * np.cos(theta)
y_local = r * np.sin(theta)

points = C[None, :] + np.outer(x_local, u_axis) + np.outer(y_local, v_axis)

# Проверка расстояний
rel = points - C[None, :]
proj_x = np.dot(rel, u_axis)
proj_y = np.dot(rel, v_axis)
radial = np.sqrt(proj_x**2 + proj_y**2)

print("Points generated:", N_points)
print("Radial distances: min =", radial.min(), "max =", radial.max())
```

```
print("Within Rc tolerance? ", np.all(radial <= Rc + 1e-12))

# Проверка равномерности по кольцам
n_bins = 50
bins = np.linspace(0, Rc, n_bins+1)
counts, _ = np.histogram(radial, bins=bins)
areas = np.pi * (bins[1:]**2 - bins[:-1]**2)
density = counts / areas

print("₩nРезультат проверки статистической равномерности:")
print("min:", counts.min(), "max:", counts.max(), "mean:", counts.mean())
print("статистика плотностей:")
print("среднее =", density.mean())
print("стандартное отклонение =", density.std())

# отобразим 300 точек
subset_idx = np.random.choice(N_points, size=300, replace=False)
pts3 = points[subset_idx]

# ---------------- FIGURE 1 ----------------
fig = plt.figure(figsize=(12, 6))

ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.scatter(pts3[:,0], pts3[:,1], pts3[:,2], s=8)

angles = np.linspace(0, 2*np.pi, 200)
rim = C[None,:] + np.outer(Rc*np.cos(angles), u_axis) +
np.outer(Rc*np.sin(angles), v_axis)
ax1.plot(rim[:,0], rim[:,1], rim[:,2], 'r-', linewidth=1.2)
ax1.set_title("Подмножество точек и окружность (3D)")
ax1.set_xlabel('X'); ax1.set_ylabel('Y'); ax1.set_zlabel('Z')
ax1.view_init(elev=25, azim=30)

ax2 = fig.add_subplot(1, 2, 2)
ax2.scatter(proj_x[subset_idx], proj_y[subset_idx], s=8)
circle = plt.Circle((0,0), Rc, edgecolor='r', facecolor='none')
ax2.add_patch(circle)
ax2.set_aspect('equal', 'box')
ax2.set_xlim(-Rc*1.1, Rc*1.1); ax2.set_ylim(-Rc*1.1, Rc*1.1)
ax2.set_title("Проекция на плоскость окружности (локальные
координаты)")
```

```
plt.tight_layout()

# ---------------- FIGURE 2 ----------------
fig2 = plt.figure(figsize=(12, 6))

# плотность по радиусу
ax3 = fig2.add_subplot(1, 2, 1)
bin_centers = 0.5*(bins[:-1] + bins[1:])
ax3.plot(bin_centers, density, marker='o', linestyle='-')
ax3.set_xlabel('радиус')
ax3.set_ylabel('точек на единицу площади')
ax3.set_title('Эмпирическая плотность вдоль радиуса (должна быть
постоянной)')
ax3.grid(True)
ax3.set_ylim(10000, max(density)*1.05)

# pdf(r) - функция плотности вероятности
ax4 = fig2.add_subplot(1, 2, 2)
hist_counts, hist_bins = np.histogram(radial, bins=100, density=True)
bin_centers2 = 0.5*(hist_bins[:-1] + hist_bins[1:])
theo = 2*bin_centers2/(Rc**2)
ax4.plot(bin_centers2, hist_counts, label='эмпирическая pdf(r)')
ax4.plot(bin_centers2, theo, '--', label='теоретическая pdf 2r/Rc^2')
ax4.set_xlabel('r')
ax4.set_ylabel('pdf')
ax4.set_title('Радиальное распределение
(эмпирическое-теоретическое)')
ax4.legend()
ax4.grid(True)


plt.tight_layout()
plt.show()

# Вывод результатов
print("₩n Контрольные проверки:")
print("- максимальный радиус <= Rc:", radial.max() <= Rc + 1e-12)
print("- коэффициент вариации:", density.std() / density.mean())
```

Для доказательства равномерности распределения точек, разобьём круг на кольца и посчитаем относительную плотность в каждом из колец.

```
Результат проверки статистической равномерности:
min: 46 max: 3966 mean: 2000.0
Статистика плотностей:
среднее = 20438.869944075584
стандартное отклонение = 724.156379883889
```

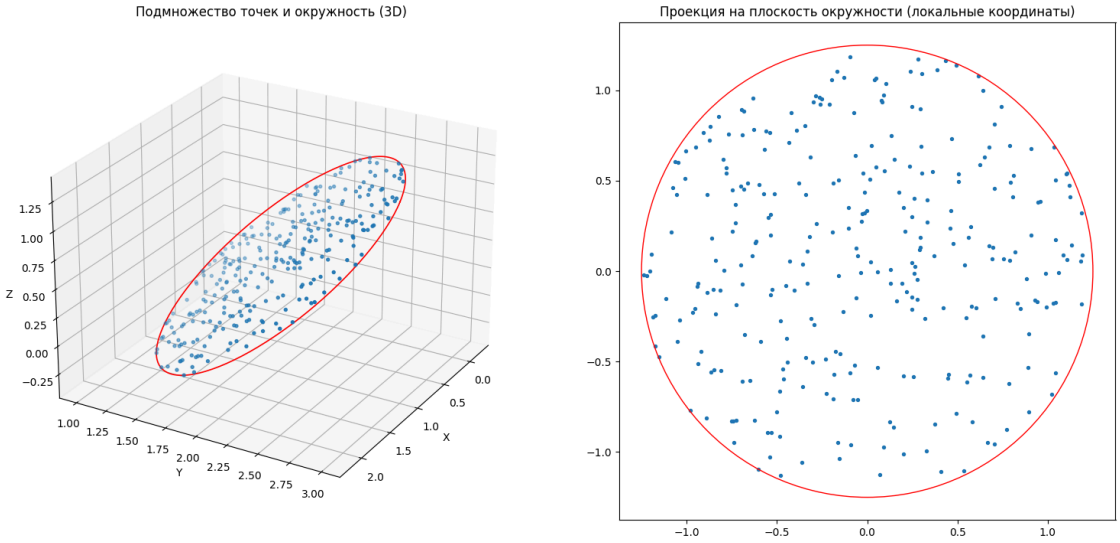


Рис. 2.2 Визуализация генерации точек в плоскости круга

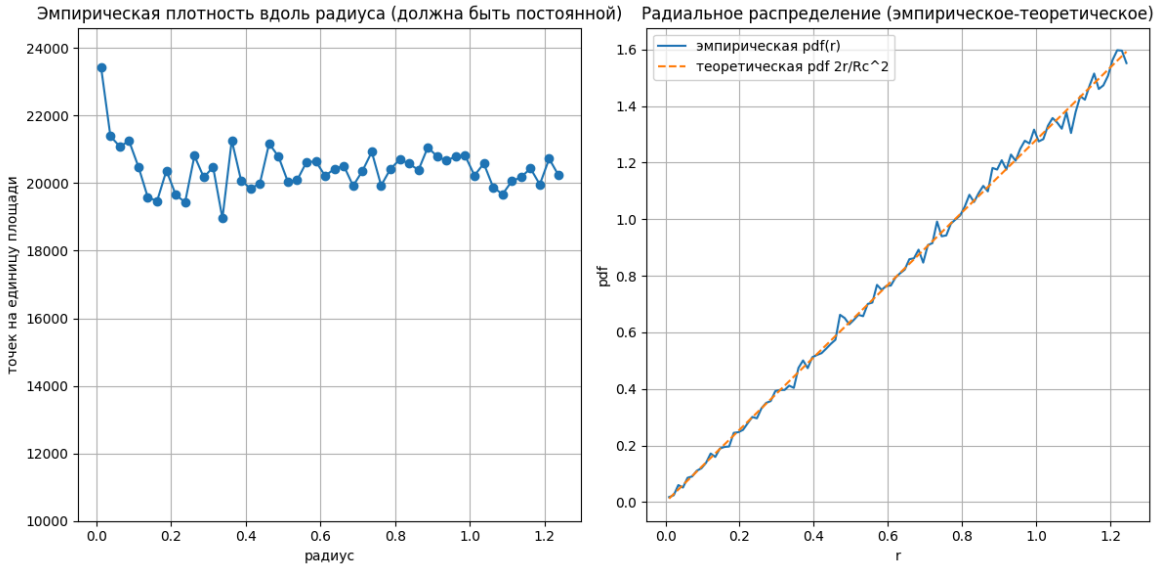


Рис. 2.3 Распределение плотностей генерируемых точек

3. Сформировать равномерное распределение случайных направлений в пространстве (точек на единичной сфере P_1, P_2, …). Число точек используемых в выборке 100000, число полученных направлений также 100000.

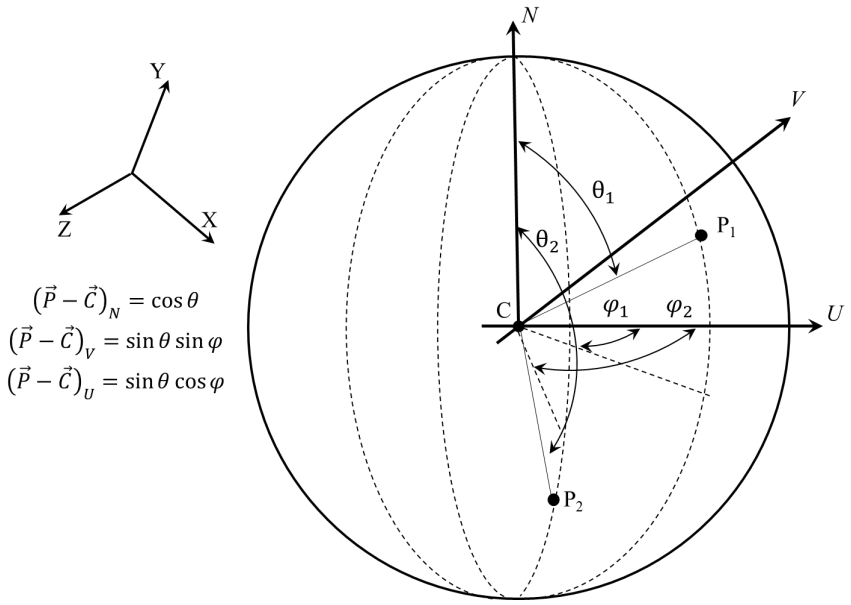


Рис. 3.1 Задание 3

Доказать, что полученное распределение равномерное и для всех выборок были сформированы направления.

Решение:

Листинг 3 Генерация равномерных направлений на сфере

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D


# 1. Генерация равномерных направлений на сфере
# ==============================================
N = 100000

# cos(theta) равномерно в [-1, 1]
u = np.random.uniform(-1, 1, N)
```

```
# phi равномерно в [0, 2π]
phi = np.random.uniform(0, 2*np.pi, N)

theta = np.arccos(u)

# Декартовы координаты (точки на сфере)
x = np.sin(theta) * np.cos(phi)
y = np.sin(theta) * np.sin(phi)
z = np.cos(theta)

points = np.vstack((x, y, z)).T  # shape = (100000, 3)

# 2. Проверка — все точки на сфере
# ============================================================
r = np.sqrt(x*x + y*y + z*z)
print("Среднее отклонение радиуса от 1:", np.mean(np.abs(r - 1)))

# 3. Проверка равномерности распределения
#    В идеальном случае гистограмма Z должна быть ровной
# ============================================================

plt.figure(figsize=(8, 4))
plt.hist(z, bins=40, density=True, color='royalblue')
plt.title("Гистограмма распределения z = cos(theta)")
plt.xlabel("z")
plt.ylabel("Плотность")
plt.grid(True)

# 4. 3D визуализация облака точек
# ============================================================
fig = plt.figure(figsize=(6, 6))
ax = fig.add_subplot(111, projection='3d')

# отображаем только часть (иначе слишком густо)
sample = points[:5000]

ax.scatter(sample[:, 0], sample[:, 1], sample[:, 2], s=2)
ax.set_title("Равномерные точки на сфере (5000 из 100000)")
ax.set_box_aspect([1, 1, 1])
ax.set_xlabel("X")
```

```
ax.set_ylabel("Y")
ax.set_zlabel("Z")

plt.show()
```

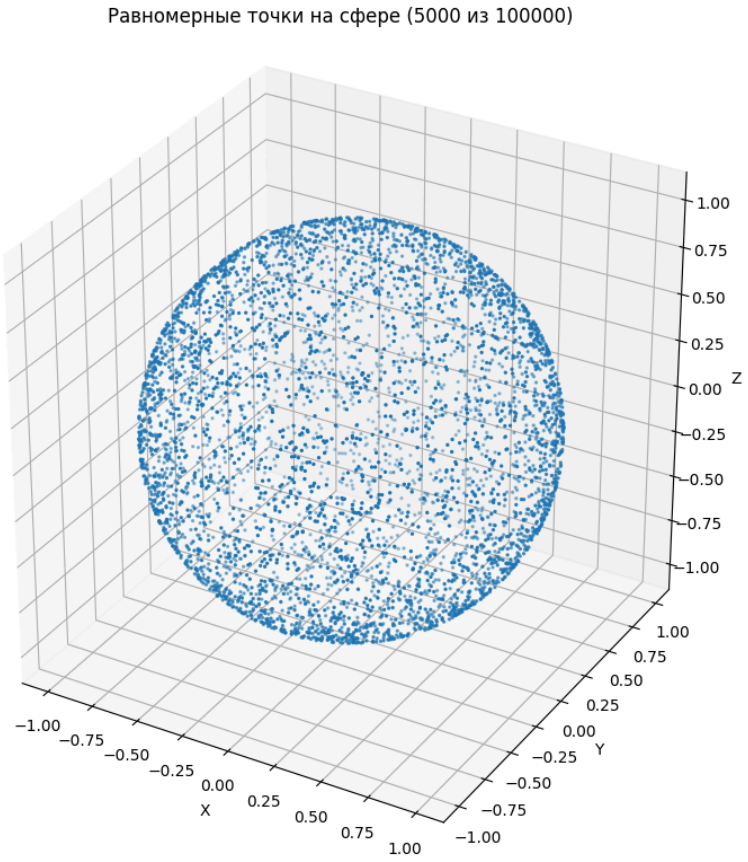


Рис. 3.2 Визуализация генерации точек на сфере

Результат проверки отклонения от радиуса

```
Среднее отклонение радиуса от 1: 2.5965896099933162e-17
```

Для проверки равномерности построим гистограмму углов отклонения. В идеальном случае гистограмма должна представлять ровную линию.

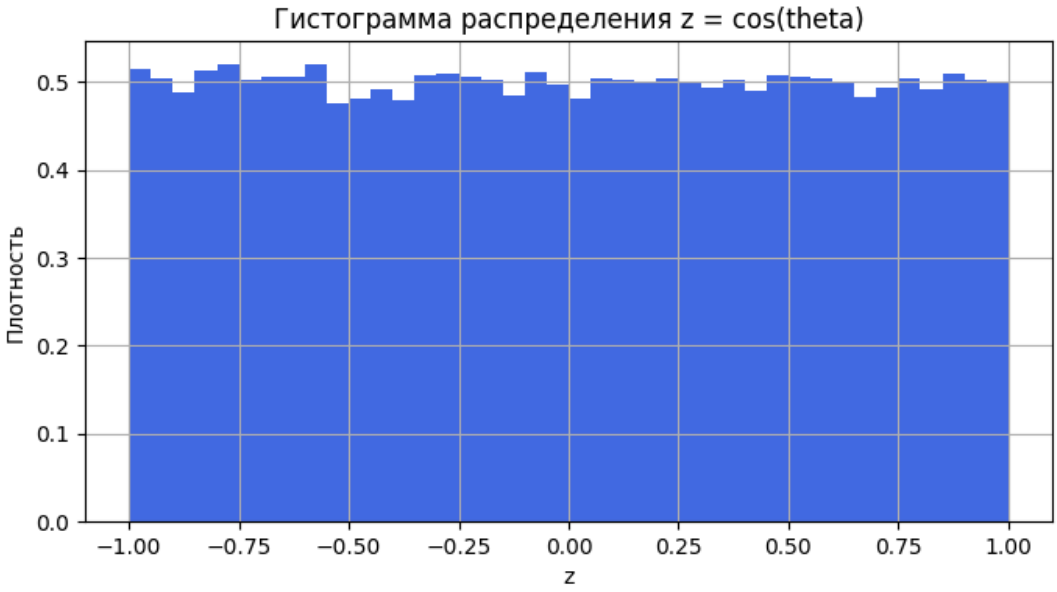


Рис. 3.3 Гистограмма распределения углов отклонения

4. Сформировать распределение случайных направлений с косинусным распределением плотности вероятности относительно вектора N (единичных векторов, ориентированных по направлению P_1 – C, P_2 – C, …, плотность вероятности пропорциональна длине вектора P – C). Распределение симметрично относительно вектора N. Число точек используемых в выборке 100000, число полученных направлений также 100000.

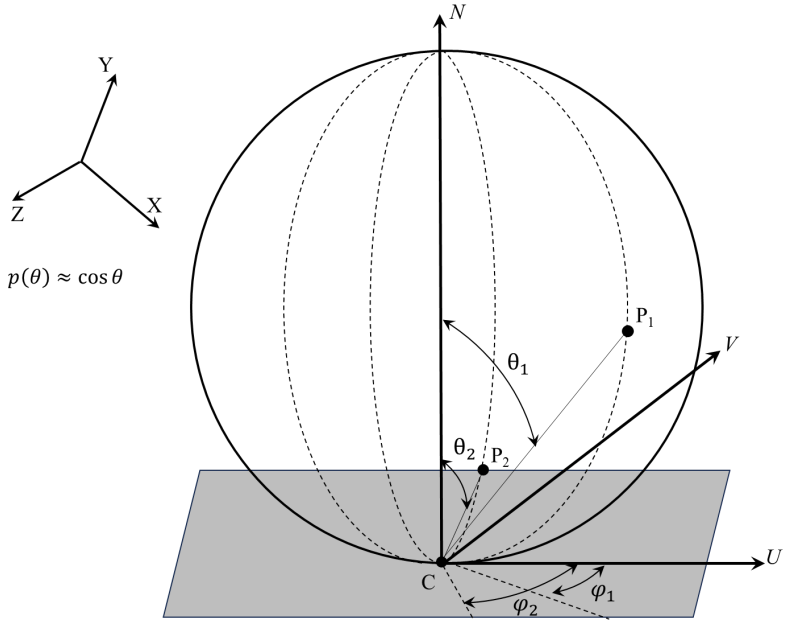


Рис. 4.1 Задание 4

Доказать, что полученное распределение косинусное и для всех выборок были сформированы направления.

Решение:

Если генерировать точки с помощью векторов, равномерно распределенных по азимуту φ от 0 до 360° и углу места θ от 0 до 180, то это не даст нам равномерного распределения точек на сфере. Это связано с тем, что кольца около полюсов меньше, чем у экватора и, при равномерном θ мы получим картинку, где точки сильнее концентрируются у полюсов.

Для равномерного распределения на сфере нужно φ — равномерный в диапазоне [0, 2π], cos(θ) — равномерный в диапазоне [-1, 1].

Листинг 4 Генерация векторов с косинусным распределением

```
import numpy as np
import matplotlib.pyplot as plt

# ----- исходные данные -------
N_points = 100000
C = np.array([0.5, -0.2, 0.3])        # точка C (южный полюс)
N_vec = np.array([0.2, 0.6, 0.76])    # ось N ()
R = 1.0                               # радиус сферы

def normalize(v, eps=1e-12):
    v = np.array(v, dtype=float)
    n = np.linalg.norm(v)
    if n < eps:
        raise ValueError("Zero length vector")
    return v / n

Nn = normalize(N_vec)

# Генерируем равномерно распределенные направления на сфере:
# cos(theta) uniformly in [-1,1],
u = np.random.rand(N_points)
mu = 2*u - 1.0          # cos(theta) uniform in [-1,1]
# phi in [0,2pi)
phi = 2*np.pi*np.random.rand(N_points)
theta = np.arccos(mu)

# переводим в декартовые координаты с осью z (0,0,1)
x = np.sqrt(1 - mu**2) * np.cos(phi)
y = np.sqrt(1 - mu**2) * np.sin(phi)
z = mu
dirs_local = np.vstack((x,y,z)).T

# поворачиваем локальную z-ось к заданной нормали
z_axis = np.array([0.0, 0.0, 1.0])
v = np.cross(z_axis, Nn)
s = np.linalg.norm(v)
```

```
if s < 1e-12:
    Rm = np.eye(3)
else:
    c = np.dot(z_axis, Nn)
    vx = np.array([[0, -v[2], v[1]],
                   [v[2], 0, -v[0]],
                   [-v[1], v[0], 0]])
    Rm = np.eye(3) + vx + vx @ vx * ((1 - c) / (s*s))

dirs = dirs_local @ Rm.T
dirs = dirs / np.linalg.norm(dirs, axis=1)[:,None]

# Points on sphere surface from C shooting outwards (C is south pole)
# We want C to be at south pole: center of sphere = C + R * Nn
center = C + R * Nn
P = center + R * dirs   # points on sphere surface

# -----Диагностика распределения------
cos_theta_wrt_N = np.dot(dirs, Nn)  # should be uniform in [-1,1]
print("cosθ: min, max:", cos_theta_wrt_N.min(), cos_theta_wrt_N.max())
print("mean cosθ (should ~0):", cos_theta_wrt_N.mean())

# Prepare slabs along axis N (z coordinate in axis frame ranges [-R, R])
z_coords = np.dot(P - center, Nn)  # should be in [-R, R]
k = 20
edges = np.linspace(-R, R, k+1)
counts, _ = np.histogram(z_coords, bins=edges)
expected = N_points / k

# Plots: два изображения с двумя диаграммами на каждом
subset = np.random.choice(N_points, size=800, replace=False)

# Первое изображение
fig1 = plt.figure(figsize=(12,5))

# Подмножество точек на сфере
ax1 = fig1.add_subplot(1,2,1, projection='3d')
ax1.scatter(P[subset,0], P[subset,1], P[subset,2], s=6)
ax1.scatter(C[0], C[1], C[2], color='red', s=80, label='C (южный
полюс)')
```

```
ax1.quiver(C[0], C[1], C[2], Nn[0], Nn[1], Nn[2], length=1.2*R,
color='green', label='N axis')
ax1.set_title('Подмножество точек на сфере')
ax1.legend()
ax1.set_box_aspect([1,1,1])

# Гистограмма cosθ к N (ожидается равномерной на интервале
[−1,1])'
ax2 = fig1.add_subplot(1,2,2)
ax2.hist(cos_theta_wrt_N, bins=60, density=True, alpha=0.7)
ax2.set_title('Гистограмма cosθ к N (ожидается равномерной на
интервале [−1,1])')
ax2.set_xlabel('cosθ')
ax2.grid(True)

plt.tight_layout()

# Второе изображение
fig2 = plt.figure(figsize=(12,5))

# распределение кол−ва точек по уровням
ax3 = fig2.add_subplot(1,2,1)
centers = 0.5*(edges[:−1] + edges[1:])
ax3.bar(range(k), counts, alpha=0.7, label='empirical counts')
ax3.hlines(expected, −0.5, k−0.5, colors='r', linestyles='−−',
label='expected = N/k')
ax3.set_title('Количество в z−уровне (равные z−срезы вдоль оси N)')
ax3.set_xlabel('индекс уровня (снизу−>вверх)')
ax3.set_ylabel('кол−во')
ax3.legend()

ax4 = fig2.add_subplot(1,2,2)
# projection to local coords (u,v) for plotting
# compute local basis used earlier
# local u_axis, v_axis from rotation matrix: pick columns of Rm for x,y axes
u_axis = Rm.T @ np.array([1,0,0])
v_axis = Rm.T @ np.array([0,1,0])
proj_x = np.dot(P − center, u_axis)
proj_y = np.dot(P − center, v_axis)
ax4.scatter(proj_x[subset], proj_y[subset], s=6)
circle = plt.Circle((0,0), R, edgecolor='r', facecolor='none')
```

```
ax4.add_patch(circle)
ax4.set_aspect('equal', 'box')
ax4.set_xlim(-R*1.1, R*1.1); ax4.set_ylim(-R*1.1, R*1.1)
ax4.set_title('Проекция точек на плоскость ортогональную N
(подмножество)')

plt.tight_layout()
plt.show()
```

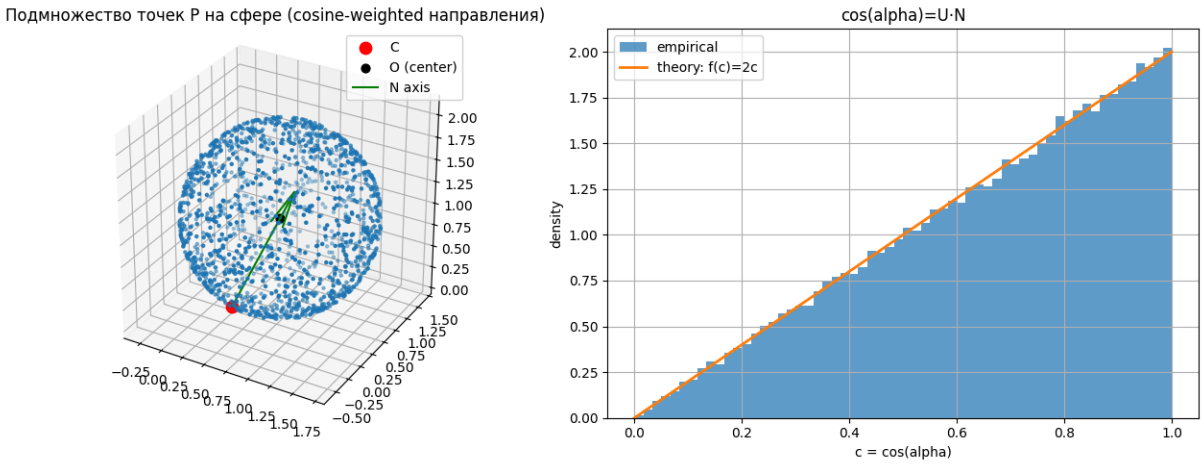


Рис. 4.2 Визуализация генерации точек на сфере

На второй диаграмме показана гистограмма распределения косинусов углов места, которая демонстрирует их равномерность во всем диапазоне [-1; 1].

Разрежем сферу на 20 равных по высоте слоев (у всех слоев одинаковая площадь поверхности) и подсчитаем, сколько в каждом секторе точек. Результат представлен на первой диаграмме рисунка 4.3. Как видим, количество достаточно равномерно и колеблется около 5000, что соответствует простому математическому расчету:

100000 (общее количество точек)/20 (число слоев) = 5000

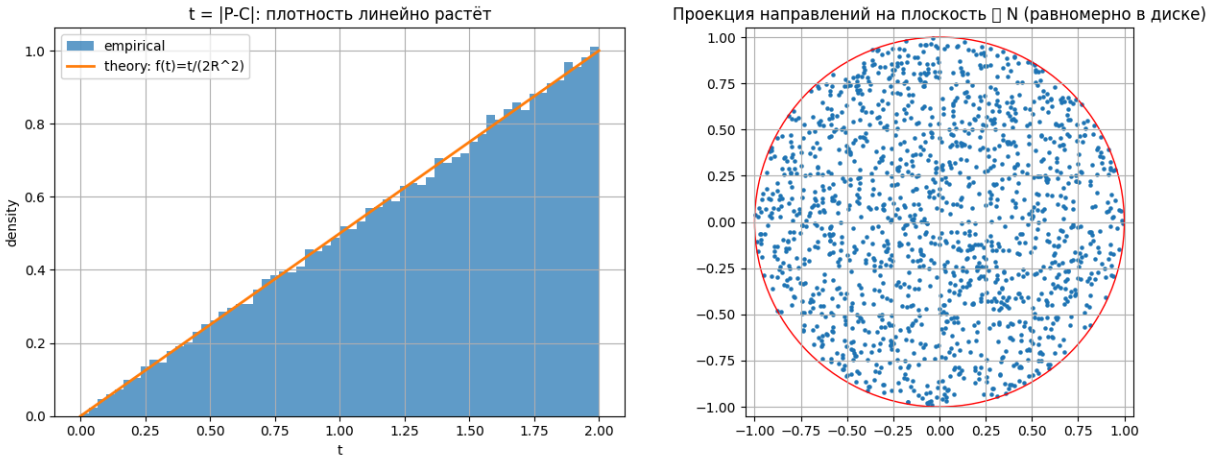


Рис. 4.2 Гистограмма распределения точек по сфере

Дополнительно показана проекция выборки точек на плоскость, ортогональную оси N.